

AprilTag Recognition

1. Experiment Objective

Use `dt_apriltags` to detect AprilTag fiducial markers and obtain their position, ID, and 3D pose in the image.

2. Experiment Principle

1. Key Terms

Term	Description
AprilTag	A visual fiducial marker system based on 2D matrix codes; each Tag carries a unique ID and supports high-accuracy 3D pose estimation
dt_apriltags	A Python AprilTag detection library (a wrapper of C++ Apriltag 3); its <code>Detector</code> class provides the detection and pose estimation API
tag36h11	The default AprilTag family; 36-bit encoding + 11 Hamming distance, offering strong resistance to false detections
3D Pose Estimation	Combined with the camera intrinsics and the actual Tag size, PnP solves the translation vector (X/Y/Z in meters) and the rotation matrix (Roll/Pitch/Yaw in degrees)
Camera Intrinsics	The focal lengths (fx/fy) and principal point (cx/cy) loaded from <code>camera.yaml</code> ; automatically scaled proportionally when the image size changes
Primary Tag Selection Strategy	<code>smallest_id</code> (smallest ID) / <code>largest</code> (largest area) / <code>nearest_center</code> (closest to center)

2. Technical Architecture

```
Terminal 1: start agent + camera + joint models
Terminal 2: start AprilTag recognition
```

Data Flow: Image subscription → image conversion → orientation correction → resize → grayscale conversion → `dt_apriltags.Detector.detect()` → 3D pose estimation combined with camera intrinsics + tag_size → draw border/center point/coordinate axes/labels → real-time display.

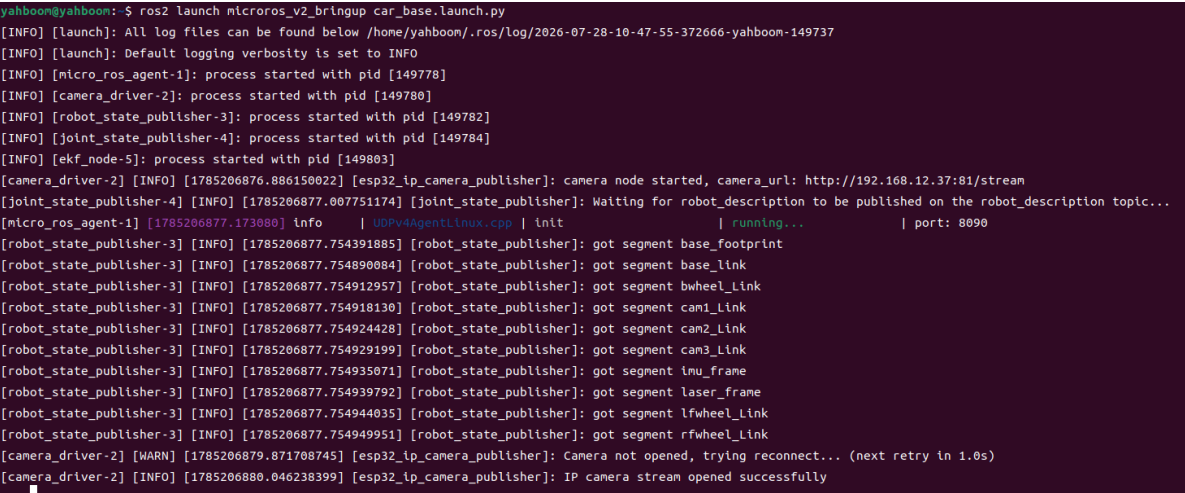
3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ Version	☒ Supported
No-PTZ Version	☒ Supported
Required Peripherals	ESP32 camera, AprilTag markers (tag36h11 family)

2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```



3. Start AprilTag Recognition (Terminal 2)

```
ros2 launch microros_v2_opencv_visual_pkg detect_apriltag.launch.py
```

4. Operation Instructions

Workflow:

- 1. Grayscale conversion → `dt_apriltags.Detector.detect()` multi-threaded scan
- 2. Returns each Tag's ID, center coordinates, 4 corners, Hamming distance, and decision margin
- 3. PnP pose estimation combined with the camera intrinsics (`camera.yaml`) and `tag_size` (default 0.025 m)
- 4. Select the primary Tag by strategy (`smallest_id` / `largest` / `nearest_center`)
- 5. Draw the Tag border (green/purple for the primary Tag), center yellow dot, `id:N` label, `Z:N.NNm` distance, and 3D coordinate axes (RGB→XYZ)

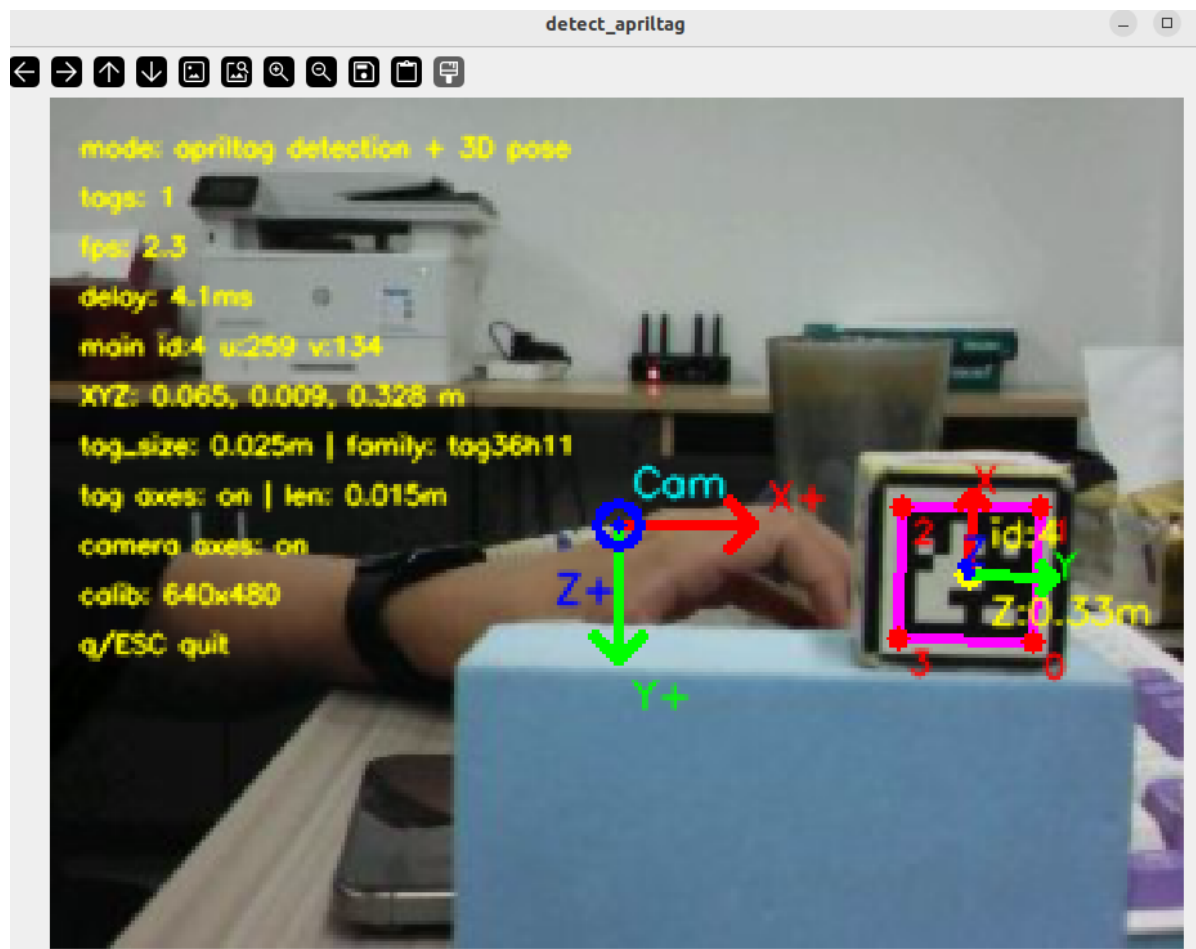
Operation	Function
<code>q</code> / <code>ESC</code>	Exit

5. How to Stop

Press `Ctrl+C` to stop the terminals one by one.

4. Experiment Observations

- **Single Tag detection:** Tag border (green polygon), 4 corner red dots numbered 0~3, center yellow dot + `id:N` + distance label
- **Multiple Tags:** Multiple Tags are marked simultaneously; the primary Tag (smallest ID/largest area) is highlighted in purple
- **3D coordinate axes:** RGB three-color axes (red=X, green=Y, blue=Z) indicate the Tag's spatial orientation
- **Real-time distance update:** the `Z:N.NNm` distance label follows the Tag movement and updates in real time



5. Program Analysis

Source code paths:

- Launch:
`~/microros_ws/src/microros_v2_opencv_visual_pkg/launch/detect_apriltag.launch.py`
- Node:
`~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/detect_apriltag.py`

1. Core Node Analysis

`DetectAprilTagNode` uses `dt_apriltags` to detect Tags and estimate their 3D pose, overlaying annotations and coordinate axes on the image.

```
# Source file:
~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/detect_apriltag.py
import dt_apriltags

class DetectAprilTagNode(Node):
    def __init__(self):
        super().__init__('detect_apriltag_node')
        self.declare_parameter('tag_family', 'tag36h11')
        self.declare_parameter('tag_size', 0.025)
        self.declare_parameter('tag_select_strategy', 'smallest_id')

        # Load camera intrinsics
        self.load_camera_intrinsics()

        # Create the dt_apriltags detector
        family = self.get_parameter('tag_family').value
        self.detector = dt_apriltags.Detector(
            families=family, nthreads=4,
            quad_decimate=1.0, quad_sigma=0.0,
            decode_sharpening=0.25)

    def image_callback(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)

        # dt_apriltags detection
        tags = self.detector.detect(
            gray, estimate_tag_pose=True,
            camera_params=(fx, fy, cx, cy),
            tag_size=self.tag_size)

        for tag in tags:
            # Draw the Tag border (connecting the 4 corners)
            corners = tag.corners.astype(int)
            cv.polylines(frame, [corners], True, color, 2)
            # Draw the center point + ID label
            cx_tag, cy_tag = int(tag.center[0]), int(tag.center[1])
            cv.circle(frame, (cx_tag, cy_tag), 4, (0, 255, 255), -1)
            cv.putText(frame, f'id:{tag.tag_id}', ...)
            # Draw the 3D coordinate axes (RGB→XYZ)
            if tag.pose_R is not None:
                self.draw_axes(frame, tag.pose_R, tag.pose_t, ...)
```

Key logic notes:

- **Automatic camera intrinsic scaling:** after reading the intrinsics from `camera.yaml`, `fx/fy/cx/cy` are automatically scaled by the ratio between the processed image size and the original calibration size
- **dt_apriltags parameters:** `nthreads=4` enables multi-threaded detection; `quad_decimate=1.0` applies no downsampling (highest accuracy);

`decode_sharpening=0.25` applies slight sharpening to improve the decoding rate

- **3D coordinate axes:** using the Tag's rotation matrix and translation vector, the 3D axis endpoints (X=0.025 m, Y=0.025 m, Z=-0.025 m) are projected onto the 2D image; the RGB colors correspond to the XYZ axis directions
- **Primary Tag selection:** in multi-Tag scenes, the primary Tag is selected by `tag_select_strategy` and highlighted in purple

2. Key Parameters

Parameter definition files:

- Launch:
`~/microros_ws/src/microros_v2_opencv_visual_pkg/launch/detect_apriltag.launch.py`
- Node:
`~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/detect_apriltag.py`

Parameter	Type	Default Value	Description
<code>tag_family</code>	string	<code>tag36h11</code>	AprilTag family
<code>tag_size</code>	float	0.025	Actual Tag side length (meters); affects distance estimation accuracy
<code>tag_select_strategy</code>	string	<code>smallest_id</code>	Primary Tag selection strategy: <code>smallest_id</code> / <code>largest</code> / <code>nearest_center</code>
<code>camera_yaml_path</code>	string	<code>config/camera.yaml</code>	Camera intrinsic configuration file

3. Node Description

Node	Function
<code>detect_apriltag_node</code>	dt_apriltags detection + 3D pose estimation + primary code selection for multiple Tags + coordinate axis visualization

6. Frequently Asked Questions

Problem	Cause	Solution
Tag not detected	Blurred Tag or insufficient light	Make sure the Tag is flat and clear, and improve the lighting
Inaccurate distance estimation	<code>tag_size</code> does not match the actual size	Change the <code>tag_size</code> parameter to match the actual Tag size
Multiple Tags interfering with each other	Tags too close together	Increase the spacing between Tags and make sure each Tag is fully visible

